

Edit distance for a run-length-encoded string and an uncompressed string [☆]

J.J. Liu ^a, G.S. Huang ^b, Y.L. Wang ^{a,b,*}, R.C.T. Lee ^b

^a Department of Information Management, National Taiwan University of Science and Technology, Taipei, Taiwan

^b Department of Computer Science and Information Engineering, National Chi Nan University, Nantou, Taiwan

Received 9 October 2006; received in revised form 4 April 2007

Available online 13 July 2007

Communicated by W.-L. Hsu

Abstract

We propose a new algorithm for computing the edit distance of an uncompressed string against a run-length-encoded string. For an uncompressed string of length n and a compressed string with M runs, the algorithm computes their edit distance in time $O(Mn)$. This result directly implies an $O(\min\{mN, Mn\})$ time algorithm for strings of lengths m and n with M and N runs, respectively. It improves the previous best known time bound $O(mN + Mn)$.

© 2007 Elsevier B.V. All rights reserved.

Keywords: Edit distance; Run-length-encoding; String compression; Algorithms

1. Introduction

Measuring the similarity or difference between two strings is fundamental to many applications. For this purpose, many measures are defined, and the *edit distance* is possibly the most popular one. On the other hand, information contents increase rapidly in recent years, and an economical way to store massive data is via data compression. However, the drawback is, in the conventional way, that one has to decompress the packed data before really using it. Therefore, if we can process compressed data directly, such as performing

string searchings or comparisons on compressed strings, we would get both advantages in saving time and space for computation.

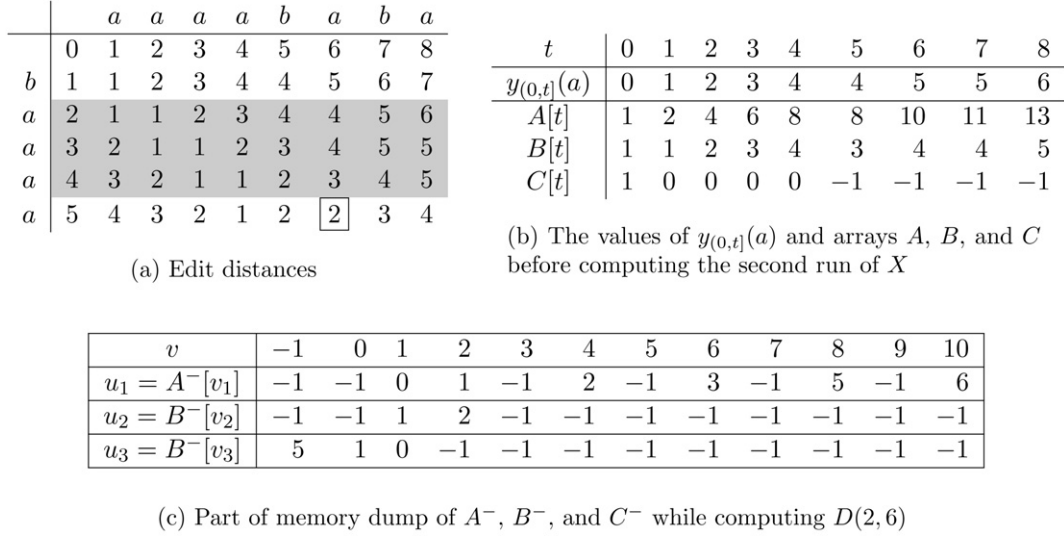
The *edit distance* [8] is defined as the minimum number of steps in order to transform one string into the other by using insertions, deletions, and substitutions on letters. The basic idea to evaluate the edit distance between any two strings is through dynamic programming. Let $X = x_1x_2x_3 \dots x_m$ and $Y = y_1y_2y_3 \dots y_n$ be two strings. We use $x_{(i-1,j]}$ to denote $x_ix_{i+1}x_{i+2} \dots x_j$, for $1 \leq i \leq j \leq m$. Similarly, $y_{(i-1,j]}$ stands for $y_iy_{i+1}y_{i+2} \dots y_j$, for $1 \leq i \leq j \leq n$. Define $\mathcal{ED}(x_{(0,i]}, y_{(0,j]})$ to be the edit distance of $x_{(0,i]}$ and $y_{(0,j]}$. It is easy to see that

$$\begin{aligned} & \mathcal{ED}(x_{(0,i]}, y_{(0,j]}) \\ &= \min \begin{cases} \mathcal{ED}(x_{(0,i]}, y_{(0,j-1]}) + 1, \\ \mathcal{ED}(x_{(0,i-1]}, y_{(0,j]}) + 1, \\ \mathcal{ED}(x_{(0,i-1]}, y_{(0,j-1]}) + t_{i,j}, \end{cases} \end{aligned} \quad (1)$$

[☆] This work was supported in part by the National Science Council of the Republic of China under Contract NSC 95-2221-E260-025.

* Corresponding author at: 1 University Road, Puli, Nantou, 54561, Taiwan, Republic of China.

E-mail address: yuelwang@ncnu.edu.tw (Y.L. Wang).

Fig. 1. An example to illustrate how to compute $\mathcal{ED}(X, Y)$.

where $t_{i,j} = 0$ if $x_i = y_j$, and 1 otherwise. And also, the boundary conditions are $\mathcal{ED}(x_{(0,i]}, y_{(0,0]}) = i$, $\mathcal{ED}(x_{(0,0]}, y_{(0,j]}) = j$ for $0 \leq i \leq m$ and $0 \leq j \leq n$, where $x_{(0,0]} = y_{(0,0]} = \varepsilon$. Clearly, the required answer $\mathcal{ED}(x_{(0,m]}, y_{(0,n]})$ can be computed in time $O(mn)$. See Fig. 1(a) for an illustration. Sophisticated algorithms for evaluating the edit distance have also been developed. There are $O(hn^2/\log n)$ time algorithm in [4] and $O(d \cdot m)$ time algorithm in [11], where $h \leq 1$ is the entropy of the text and d is the edit distance of the two considered strings and assume that $n \geq m$, respectively. For more discussions, please refer to [2,5–7,10].

Run-length encoding (abbreviated RLE) uses a simple idea to compress strings. It divides a string into several *runs*, each run consists of identical letters, and then represents the string by consecutive pairs of the representative letter and the length of the corresponding run. For example, the RLE of string *bdcccaaaaa* is $b^1 d^1 c^3 a^6$. For more details on RLE, please see [9].

In this paper, we consider two strings X and Y . The length of X is m and it has M runs in its RLE. Similarly, Y has n letters and N runs. Bunke [3], Arbell [1] and Crochemore [4] proposed an $O(mN + Mn)$ time algorithm for finding the edit distance of two RLE strings. We improve this time bound to $O(\min\{mN, Mn\})$ by computing the edit distance of an RLE string against an uncompressed string. In the rest of this paper, without loss of generality, we always assume that X is the RLE string and Y is the uncompressed string.

2. Main results

We briefly explain the basic idea in our algorithm. For a string $X = x_1 \dots x_m$ where x_i 's are letters, we use $X_1 \dots X_M$ to denote its RLE and X_i to be its i th run. Let $X_{(i,j]}$ be $X_{i+1} \dots X_j$. Another uncompressed string $Y = y_1 \dots y_n$. In order to shorten the notation, we use $D(i, j)$ as a macro for $\mathcal{ED}(X_{(0,i]}, y_{(0,j]})$, and left the general notation $\mathcal{ED}$ for other situations. From Eq. (1), it is not difficult to see that $D(i, j)$ can be computed recursively by

$$D(i, j) = \min_{0 \leq u \leq j} \{D(i-1, u) + \mathcal{ED}(X_i, y_{(u,j]})\}. \quad (2)$$

Note that in this formula, we break $X_{(0,i]}$ into $X_{(0,i-1]}$ and X_i for $1 \leq i \leq M$, and try to find the corresponding breaking position u in $y_{(0,j]}$ for $0 \leq u \leq j \leq n$. By applying this recursion, we will show how to skip entries in the dark area as shown in Fig. 1(a) in which the top of the table is the uncompressed string and on the left side is the RLE string. This makes the computation depend on M , but not m . The main question is how to evaluate Eq. (2) quickly. The value of the second term $D(i-1, u)$ can be retrieved in constant time from a bookkeeping table. We will show later that the third term $\mathcal{ED}(X_i, y_{(u,j]})$ can also be evaluated in constant time. We also propose a simple data structure that can support to determine the required u for Eq. (2) in constant time.

Let $\tau \in \Sigma$ be a letter in Y . The numbers of the occurrences of τ in substrings $y_{(p,q]}$ is denoted by $y_{(p,q]}(\tau)$. In addition, $y_{(p,q]}(\tau) = y_{(0,q]}(\tau) - y_{(0,p]}(\tau)$. The following lemma implies that once we have pre-computed

Table 1
Testing criteria for determining $D(i, j)$ via solving t

Rule	Precomputation, $0 \leq t \leq n$	Condition	Allowed range for t
C1	$A[t] = D(i-1, t) + y_{(0,t]}(\tau)$	$k + y_{(0,j]}(\tau) - \ell_i$	$\max\{j - \ell_i, 0\} \leq t \leq j$
C2	$B[t] = D(i-1, t) + y_{(0,t]}(\tau) - t$	$k + y_{(0,j]}(\tau) - j$	$0 \leq t \leq j - \ell_i$ and $y_{(t,j]}(\tau) \leq \ell_i$
C3	$C[t] = D(i-1, t) - t$	$k - j + \ell_i$	$0 \leq t \leq j$ and $\ell_i \leq y_{(t,j]}(\tau)$

$y_{(0,t]}(\tau)$ for $0 \leq t \leq n$, the third term $\mathcal{ED}(X_i, y_{(u,j]})$ in Eq. (2) can be evaluated in constant time.

Lemma 1. Let τ be the representative letter in run X_i and $\ell_i = |X_i|$ for $1 \leq i \leq M$. Then

$$\mathcal{ED}(X_i, y_{(u,j]}) = \max\{\ell_i, j - u\} - \min\{\ell_i, y_{(u,j]}(\tau)\}. \quad (3)$$

Proof. The second term $\max\{\ell_i, j - u\}$ comes from the fact that we have to delete some letters from a longer string in order to transform it into a shorter one and place mismatches for the remaining part. The third term $\min\{\ell_i, y_{(u,j]}(\tau)\}$ states that we can match at most $\min\{\ell_i, y_{(u,j]}(\tau)\}$ letters and thus reduce the number of mismatches by this quantity. $\square$

Indeed, Eq. (3) in Lemma 1 can be represented by the following three cases after considering the possible values of ℓ_i , $j - u$, and $y_{(u,j]}(\tau)$:

$$\mathcal{ED}(X_i, y_{(u,j]}) = \begin{cases} \ell_i - y_{(u,j]}(\tau) & \text{if } \ell_i \geq j - u, \\ j - u - y_{(u,j]}(\tau) & \text{if } y_{(u,j]}(\tau) \leq \ell_i \leq j - u, \\ j - u - \ell_i & \text{if } \ell_i \leq y_{(u,j]}(\tau), \end{cases} \quad (4)$$

where $0 \leq u \leq j \leq n$ and $1 \leq i \leq M$. It is clear that $y_{(p,q]}(\tau) = y_{(0,q]}(\tau) - y_{(0,p]}(\tau)$ can be evaluated in $O(1)$ time for any $p \leq q$, provided the pre-computation of $y_{(0,t]}(\tau)$ for $0 \leq t \leq n$. The pre-computation can be finished in total $O(n)$ time for each letter τ by a linear scan of Y . Hence, Eq. (3) can be done in $O(1)$ time.

The parameter u in Eq. (2) ranges between $[0, j]$. Even though we can evaluate the expression of u inside Eq. (2) in constant time, this is not enough since we still have to determine which u makes the minimum. In [11], Ukkonen gave a relation between $D(i, j)$ and $D(i, j - 1)$ as follows.

Lemma 2. (See [11].) $D(i, j) - D(i, j - 1) \in \{-1, 0, 1\}$.

By Lemma 2, the possible values of $D(i, j)$ are $D(i, j - 1) - 1$, $D(i, j - 1)$, or $D(i, j - 1) + 1$. For each of the possible values, say k , we can obtain

$$k = D(i - 1, u) + \mathcal{ED}(X_i, y_{(u,j]}) \quad (5)$$

if it exists, by dropping min from Eq. (2). The smallest k that leads to a corresponding u in Eq. (5) is the actual $D(i, j)$.

Now substitute Eq. (4) into Eq. (5), and then replace $y_{(u,j]}(\tau)$ by $y_{(0,j]}(\tau) - y_{(0,u]}(\tau)$. Note that each condition, namely the if term, in Eq. (4) is rearranged such that the middle term of one inequality is only with u . Moreover, the relations of ℓ_i and $y_{(u,j]}(\tau)$ in the second and third conditions remain unchanged. Finally, separate terms depending on u to the left-hand side of the equality and others to the right-hand side. We get the following constraints:

- (C1) $D(i - 1, u) + y_{(0,u]}(\tau) = k + y_{(0,j]}(\tau) - \ell_i$ whenever $\max\{j - \ell_i, 0\} \leq u \leq j$;
- (C2) $D(i - 1, u) + y_{(0,u]}(\tau) - u = k + y_{(0,j]}(\tau) - j$ whenever $0 \leq u \leq j - \ell_i$ and $y_{(u,j]}(\tau) \leq \ell_i$;
- (C3) $D(i - 1, u) - u = k - j + \ell_i$ whenever $0 \leq u \leq j$ and $\ell_i \leq y_{(u,j]}(\tau)$.

Let A , B , and C be arrays to store the values on the left-hand sides of C1, C2, and C3, respectively, for $0 \leq u \leq n$. It is easy to see that A , B , and C can be built in time $O(n)$ for fixed i , $i \geq 1$, and τ (see column 2 of Table 1 and an illustration in Fig. 1(b)).

Our main idea is as follows. At first, using the expressions in the right-hand side of constraints C1, C2, and C3 (see column 3 of Table 1) one by one with the possible values of k also one by one to get a value, say r . To check whether r is in the corresponding precomputed arrays A , B , or C , we need to scan the corresponding array in order to find an index, say t , whose element value in the array is r . And, finally, we have to check whether t is in the specific range as specified in column 4 of Table 1. If the answer is yes, then $u = t$ and $D(i, j) = k$; otherwise, try another pair of constraint and k . However, in the above process, scanning an array will take $O(n)$ time on each trail. To find an efficient way, our approach is to build inverted lists A^- , B^- and C^- for arrays A , B and C , respectively, such that $A^-[v_1] = t$, $B^-[v_2] = t$, or $C^-[v_3] = t$ if and only if there is an index t such that $A[t] = v_1$, $B[t] = v_2$, or $C[t] = v_3$. $A^-[v_1]$, $B^-[v_2]$, and $C^-[v_3]$ are defined as follows.

$A^-[v_1] = \max\{u \mid A[u] = v_1 \text{ for } 0 \leq u \leq j\},$
 for $-n \leq v_1 - A[0] \leq 2n;$
 $B^-[v_2] = \max\{u \mid B[u] = v_2 \text{ for } 0 \leq u \leq j - \ell_i\},$
 for $-2n \leq v_2 - B[0] \leq n;$
 $C^-[v_3] = \min\{u \mid C[u] = v_3 \text{ for } 0 \leq u \leq j\},$
 for $-2n \leq v_3 - C[0] \leq 0,$
 with the initial value -1 .

Note that A^- , B^- , and C^- depend on j (index for y_j) and i (index for X_i). We omit the dependency because they can be updated incrementally in $O(1)$ time as shown in the procedure `UpdateInvertedLists()`.

Procedure `UpdateInvertedLists(int i, int j)`

```

1  begin
2     $A^-[A[j]] = j;$ 
3    if  $j \geq \ell_i$  then  $B^-[B[j - \ell_i]] = j - \ell_i;$ 
4    if  $C^-[C[j]] = -1$  then  $C^-[C[j]] = j;$ 
5  end

```

Then we have the following lemma:

Lemma 3. *Fixed i , the following claims are true while computing $D(i, j)$:*

- (1) Arrays A^- , B^- , and C^- can be updated incrementally in constant time when j increases from 0 to n .
- (2) The ranges of $A[t]$, $B[t]$, and $C[t]$ for $0 \leq t \leq n$ are bounded within $A[0] + [-n, 2n]$, $B[0] + [-2n, n]$, $C[0] + [-2n, 0]$, respectively.
- (3) Let $k \in \{D(i, j - 1) - 1, D(i, j - 1), D(i, j - 1) + 1\}$, $\ell_i = |X_i|$, and $\tau \in X_i$. Eq. (5) has a solution if and only if at least one of the following conditions hold:
 - (a) $-n \leq v_1 - A[0] \leq 2n$ and $\max\{j - \ell_i, 0\} \leq A^-[v_1]$,
 - (b) $-2n \leq v_2 - B[0] \leq n$, $0 \leq B^-[v_2]$ and $y_{(u', j]}(\tau) \leq \ell_i$, or
 - (c) $-2n \leq v_3 - C[0] \leq 0$, $0 \leq C^-[v_3]$ and $\ell_i \leq y_{(u', j]}(\tau)$,
 where $v_1 = k + y_{(0, j]}(\tau) - \ell_i$, $v_2 = k + y_{(0, j]}(\tau) - j$, and $v_3 = k - j + \ell_i$.

Proof. Firstly, we prove the first claim. As for stage j , we only need to consider the contribution of $A[j]$, $B[j - \ell_i]$, and $C[j]$ to A^- , B^- , and C^- , respectively. Clearly the update operations are local and can be finished in $O(1)$ time as in `UpdateInvertedLists()`.

The second claim is proved as follows. Consider $A[t + 1] - A[t]$, $B[t + 1] - B[t]$ and $C[t + 1] - C[t]$ for $0 \leq t < n$. From Lemma 2, we know $D(i - 1, t + 1) - D(i - 1, t) \in [-1, 1]$. Clearly, $y_{(0, t+1]}(\tau) - y_{(0, t]}(\tau) \in [0, 1]$, and $(t + 1) - t \in [1, 1]$. Hence we have $A[t + 1] - A[t] \in [-1, 1] + [0, 1] = [-1, 2]$, $B[t + 1] - B[t] \in [-1, 1] + [0, 1] - [1, 1] = [-2, 1]$, and $C[t + 1] - C[t] \in [-1, 1] - [1, 1] = [-2, 0]$. Therefore the claim holds.

The third claim directly comes from C1, C2, and C3. In case C1, we know that there exists u such that $A[u] = v$ and $\max\{j - \ell_i, 0\} \leq u \leq j$. From the definition of A^- , we know $A[A^-[v_1]] = A[u]$ and $A^-[v_1] \geq u$. Combining them, we get $\max\{j - \ell_i, 0\} \leq u \leq A^-[v_1] \leq j$, which implies that $A^-[v_1]$ is an alternative of u . The other direction comes immediately from the definition of A^- . The proofs for cases (b) and (c) are similar. $\square$

3. The algorithm

The complete algorithm is given in Algorithm 3. It calls externally `UpdateInvertedLists()` and `TestGuessing()`. Function `TestGuessing()` is written according to columns 3 and 4 of Table 1. In Algorithm 3, lines 4–6 initialize the arrays A , B , C for the i th row, and then lines 7–9 initialize the inverted lists A^- , B^- , and C^- . Line 11 updates A^- , B^- , and C^- for the entry $D(i, j)$. Line 13 tries to test if $D(i, j)$ equals to $D(i, j - 1) - 1$, and line 14 tries to test if $D(i, j)$ equals to $D(i, j - 1)$. If neither cases hold, we get $D(i, j) = D(i, j - 1) + 1$ in line 15. It is evident that the time complexity is $O(Mn)$ since the outer loop (lines 2–18) executes M times, the inner loops (lines 4–6, 7–9, 10–17) run $O(n)$ times, and each step (lines 3, 5, 8, 11–16) takes only $O(1)$ time.

Function `TestGuessing(int k, int i, int j)`

```

1  begin
2     $v_1 = k + y_{(0, j]}(\tau) - \ell_i;$ 
3    if  $-n \leq v_1 - A[0] \leq 2n$  then
4       $u_1 = A^-[v_1];$ 
5      if  $(j - \ell_i \leq u_1)$  and  $(0 \leq u_1)$  then return True;
6    end
7     $v_2 = k + y_{(0, j]}(\tau) - j;$ 
8    if  $-2n \leq v_2 - B[0] \leq n$  then
9       $u_2 = B^-[v_2];$ 
10     if  $(0 \leq u_2)$  and  $(y_{(u_2, j]}(\tau) \leq \ell_i)$  then return True;
11   end
12    $v_3 = k - j + \ell_i;$ 
13   if  $-2n \leq v_3 - C[0] \leq 0$  then
14      $u_3 = C^-[v_3];$ 

```

```

15     if  $(0 \leq u_3)$  and  $(\ell_i \leq y_{(u_3, j]}(\tau))$  then return True;
16   end
17   return False;
18 end

```

Algorithm 3. The complete algorithm

```

1  for  $t = 0$  to  $n$  do  $D(0, t) = t$ ;
    // Process row  $i$ 
2  for  $i = 1$  to  $M$  do
3     $\tau \in X_i$ ;  $\ell_i = |X_i|$ ;  $D(i, 0) = D(i - 1, 0) + \ell_i$ ;
    // Initialize arrays  $A, B, C$  for row  $i$ 
4    for  $t = 0$  to  $n$  do
5       $A[t] = D(i - 1, t) + y_{(0, t]}(\tau)$ ;  $B[t] = A[t] - t$ ;
       $C[t] = D(i - 1, t) - t$ 
6    end
    // Initialize arrays  $A^-, B^-, C^-$ 
    for row  $i$ 
7    for  $t = 0$  to  $3n - 1$  do
8       $A^-[A[0] - n + t] = B^-[B[0] - 2n + t]$ 
       $= C^-[C[0] - 2n + t] = -1$ 
9    end
    // Fill row  $i$ 
10   for  $j = 0$  to  $n$  do
11     UpdateInvertedLists ( $i, j$ );
12     if  $j > 0$  then
13       // Guess  $D(i, j)$  from  $D(i, j) - 1$ ,
14        $D(i, j), D(i, j) + 1$ 
15       if TestGuessing( $D(i, j) - 1, i, j$ ) then
16          $D(i, j) = D(i, j) - 1$ ;
17       else if TestGuessing( $D(i, j), i, j$ ) then
18          $D(i, j) = D(i, j)$ ;
19       else  $D(i, j) = D(i, j) + 1$ 
20     end
21   end
22 end

```

We use an example to illustrate Algorithm 3. In Fig. 1(a), we are interested in calculating $D(2, 6)$ which is embraced in a box located at the bottom line. Note that $Y = aaaababa$, $X = b^1a^4$, $X_1 = b$, $X_2 = a^4$, $\ell_2 = 4$, $i = 2$, $j = 6$, $\tau = a$, $M = 2$, and $n = 8$. The contents for arrays $A[t]$, $B[t]$, and $C[t]$ for $0 \leq t \leq 8$ are listed in Fig. 1(b), which are obtained from lines 4 to 6. Since $D(2, 5) = 2$, we know $D(2, 6) \in \{2 - 1, 2, 2 + 1\}$ from Lemma 2. We give the first trial of $D(2, 5)$ with value 1 in line 13, and TestGuessing returns False. In the trial with value 2 in line 14, fortunately, TestGuessing returns True. Therefore, $D(2, 6) = 2$. We summarize our result as the following theorem.

Theorem 1. The Edit distance of an RLE string X and an uncompressed string Y can be computed in $O(Mn)$ time by Algorithm 3.

For any two strings X and Y , we can choose to compress X or compress Y . Therefore, we conclude that their edit distance can be computed in $O(\min\{mN, Mn\})$ time.

4. Conclusion

In this paper, we propose an $O(\min\{Mn, Nm\})$ time algorithm for solving the edit distance of an RLE string and an uncompressed string. We point out that the techniques used here can be generalized. Let ι be the insertion cost, δ be the deletion cost, and σ be the substitution cost. Let $g = \gcd(\iota, \delta, \sigma)$. If ι/g , δ/g , and σ/g are all $O(1)$, similar nice properties in Lemmas 1–3 also hold.

As like other algorithms, it is obvious that our algorithm can be parallelized in a PRAM EREW model by using $O(\min\{M, n\})$ processors in $O(M + n)$ time. This technique computes the entries in the $D(i, j)$ table diagonally.

References

- [1] O. Arbell, G.M. Landau, J.S.B. Mitchell, Edit distance of run-length encoded strings, *Information Processing Letters* 83 (2002) 307–314.
- [2] Z. Bar-Yossef, T.S. Jayram, R. Krauthgamer, R. Kumar, Approximating edit distance efficiently, in: *Proceedings of the 45th Annual IEEE Symposium on Foundations of Computer Science*, 2004.
- [3] H. Bunke, J. Csirik, An improved algorithm for computing the edit distance of run length coded strings, *Information Processing Letters* 54 (1995) 93–96.
- [4] M. Crochemore, G.M. Landau, M. Ziv-Ukelson, A subquadratic sequence alignment algorithm for unrestricted scoring matrices, *SIAM Journal of Computing* 32 (6) (2003) 1654–1673.
- [5] Z. Galil, K. Park, Data structures and algorithms for approximate string matching, *Journal of Complexity* 4 (1988) 33–72.
- [6] Z. Galil, K. Park, An improved algorithm for approximate string matching, *SIAM Journal on Computing* 19 (6) (1990) 989–999.
- [7] D. Gusfield, *Algorithms on Strings, Trees, and Sequences*, Cambridge University Press, 1997.
- [8] V.I. Levenshtein, Binary codes capable of correcting, deletions, insertions and reversals, *Soviet Physics Doklady* 10 (1996) 707–710.
- [9] K. Sayood, E. Fow (Eds.), *Introduction to Data Compression*, second ed., Morgan Kaufmann, Los Altos, CA, 2000.
- [10] J. Tarhio, E. Ukkonen, Approximate Boyer–Moore string matching, *SIAM Journal on Computing* 22 (2) (1993) 243–260.
- [11] E. Ukkonen, Algorithms for approximate string matching, *Information and Control* 64 (1985) 100–118.